

## Naming

Names are used to specify (belirtmek) entities in a distributed system.

To use an entity, we access it through an **access point**.

Access points are named by an **address**.

### Note

A **location-independent name** does not depend on the address of the access point.

So, the entity can change location, but the name can stay the same.

## Identifiers

### Pure name

A **pure name** has no meaning.

It is only a random string and is used for comparison.

### Identifier

An **identifier** is a name with special properties:

1. It refers to at **most one entity**.
2. Each entity has at **most one identifier**.
3. It always refers to the same entity and is never reused.

### Observation

An identifier does not have to be a pure name.

It may also have content.

## Flat Naming -- Locating an Entity

## Broadcasting

### Broadcast the ID

Broadcast the **ID** and ask the entity to return its current address.

- It only works well in local-area networks.
- All processes must listen to location requests.

## Address Resolution Protocol (ARP)

**ARP** maps an IP address to a physical **MAC address** in a LAN.

To find the MAC address, it broadcasts: **"Who has this IP address?"** ex : ARP

## Forwarding Pointers

### Main Idea

When an entity moves, it leaves a pointer to its next location.

- Clients can find the entity by following the pointer chain.
- When the current location is found, the client reference can be updated.

- This has scalability problems.

## Problems

- Long chains are not fault tolerant.  $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$
- Following many pointers increases network latency.

# Home-based Approaches

## Single-tiered Scheme

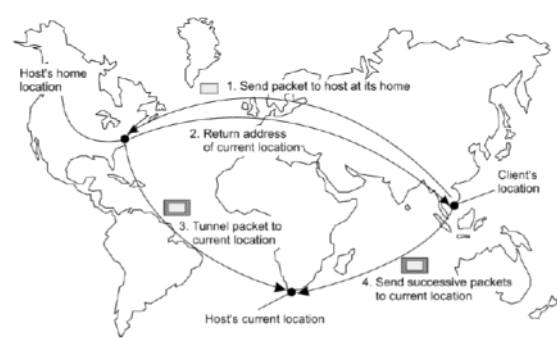
A **home** keeps track of where the entity is.

- The entity's **home address** is registered in a naming service.
- The home stores the entity's **foreign address**.
- The client contacts the home first, then goes to the foreign location.

## The Principle of Mobile IP

Mobile IP uses the **home location** to find the host's current location.

1. The client sends a packet to the host's home.
2. The home returns the host's current address.
3. The packet is tunneled to the current location.
4. After that, packets are sent to the current location.



## Illustrative: Chord

### ! Logical Ring

Chord organizes many nodes in a **logical ring**.

- Each node has a random **m-bit identifier**.
- Each entity has a unique **m-bit key**.
- Key **k** belongs to the node with the smallest **id ≥ k**.
- This node is called the **successor succ(k)**.

## Nonsolution

Each node **only knows its neighbor** and searches linearly around the ring. This is simple, but slow.

- Node **p** means the node with identifier **p**.

Örnek:

Node IDs: 3, 8, 15, 22  
Key: 10

10'dan büyük/eşit ilk node 15 olduğu için:

$\text{successor}(10) = 15$

Yani key 10, node 15'te tutulur.

Each key is stored at its successor node.

### ! Chord Finger Tables

## Solution

Finger tables are used to make searching faster in the ring.

Each node **p** does not only keep its neighbor.

It also keeps some farther nodes in its **finger table**.

$$FT_p[i] = \text{succ}(p + 2^{i-1})$$

This means the table **stores nodes at different distances from p**.

So, the search can jump to farther nodes instead of moving one by one.

$$FT_p[i] = \text{succ}(p + 2^{i-1})$$

The **i-th entry** points to the first node that is at least **2<sup>i-1</sup>** after node **p**.

Lookup cost:  $O(\log N)$  with finger table

$O(N)$  without Finger Table

# Exploiting Network Proximity

## Problem:

The logical overlay may cause **erratic message transfers**.  
Node **p** and node **succ(p + 1)** may be logically close, but physically far away.

## Solutions

### Topology-aware node assignment:

Nodes close in ID space should also be close in the real network.  
This can be very difficult.

### Proximity routing:

Keep more than one possible successor and forward to the closest one.

### Example:

In Chord, node **p** can store extra pointers to other nodes in the same interval.

### Proximity neighbor selection:

When choosing a neighbor, pick the physically closest one.

# Hierarchical Location Services (HLS)

## Basic Idea

HLS builds a large search tree.  
The network is divided into **hierarchical domains**.  
Each domain has a separate **directory node**.

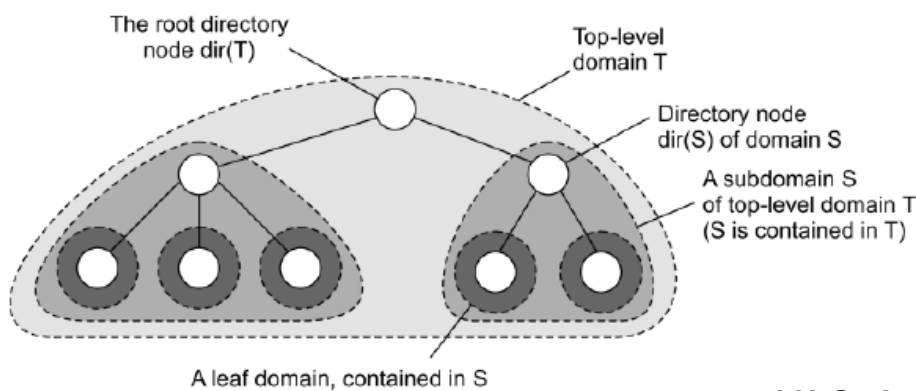
## Principle

- The root directory node is at the top.
- Top-level domains are divided into subdomains.
- Leaf domains are the lowest-level domains.

- The address of entity **E** is stored in a leaf or intermediate node.
- Intermediate nodes keep pointers only to child subtrees that contain the entity's address.
- The root knows about all entities.

## Two Addresses

If entity **E** has addresses in different leaf domains, HLS stores pointers to both locations.  
So, the entity can be found through the tree.



## HLS: Lookup Operation

- Lookup starts at the local leaf node.
- If a node knows about entity **E**, it follows the downward pointer.
- If it does not know **E**, the request goes up to the parent.
- Upward lookup stops at the root.

## Looking Up a Location

The request first goes up until it finds a node that knows **E**.  
Then it follows pointers down to the correct leaf domain.

## HLS: Insert Operation

An insert request is sent upward until it reaches the first node that already knows entity **E**.

To add a new address, the request goes up first.  
Then, pointer records are created downward to the leaf node.

# Can an HLS Scale?

HLS can scale because **logical design** and **physical implementation** are separated.  
The root node may logically track **all identifiers**, but this does not mean one physical server stores everything.  
Instead, logical location servers can be distributed over different physical hosts.  
Each host can run one or more location servers.  
Entities can also be mapped to different servers.  
So, the storage load is shared by many hosts, not only by a single root server.

# Security in Flat Naming

## Basic Idea

In flat naming, we need security measures.  
Because we must be sure that a flat name returns the correct entity.  
There are two main solutions:

- Secure the link between the **identifier** and the **entity**.
- Secure the **name-resolution process**.



## Self-Certifying Names

A self-certifying name helps us verify the entity.  
For read-only entities, the identifier can be made from the data:  
**id(entity) = hash(data)**  
If the data changes, the hash also changes.  
So, we can understand if the entity is wrong or modified.  
For other entities, the identifier can be the public key:  
**id(entity) = public key(entity)**  
Then, a **digital signature** can be used to verify the entity.

## Name-Resolution Security

Securing the full name-resolution process is more difficult.  
This topic is discussed later with secure DNS.

# Name Space

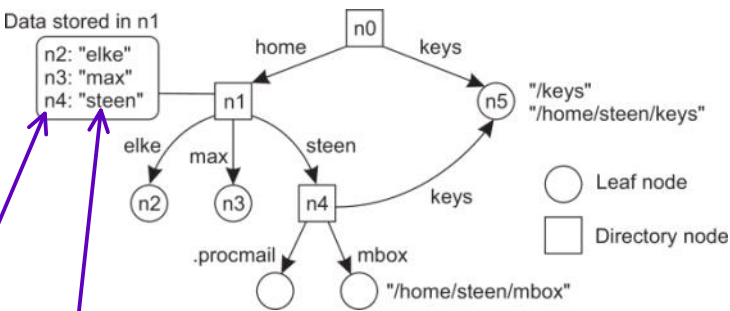
## Naming Graph

A **name space** can be represented as a naming graph.

- A **leaf node** represents a named entity.
- A **directory node** refers to other nodes.
- A directory node stores pairs like **node identifier + edge label**.

Ex:

A path like **/home/steen/keys** is found by following labels in the graph.



# Name Space

## Attributes in Nodes

A node can store different attributes:

- Entity type
- Entity identifier
- Entity address
- Nicknames



## Note

Directory nodes can also **have attributes**.  
They do not only **store directory tables**.

## Name Resolution

### Problem

To resolve a name, we need a directory node.

The question is: how do we find the first directory node?

### Closure Mechanism

A **closure mechanism** decides where name resolution starts.

Examples:

- A website name starts at a **DNS name server**.
- A file path starts at a **local file server**.

### Note

The start point cannot be fully explicit. We **need some implicit context** to start name resolution.

## Name Linking

### Hard Link

A **hard link** is a path name.

It is resolved by following a fixed path in the naming graph. `/home/steen/keys`

### Soft Link

A **soft link** lets a node contain the name of another node.

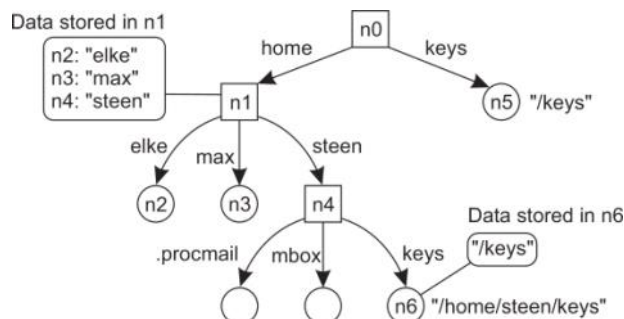
Process:

- Resolve node **N** first.
- Read the content of **N**.
- Continue name resolution with the new name.

### Observation

In soft links, name resolution reads the content of a node.

Then it continues with the name stored inside that node.



### Symbolic Link

A **symbolic link** works like a shortcut.

In the graph, node **n5** is the real target.

Its real name is only:

**/keys**

Node **n6** is not the real target.

It stores another name inside it:

**"/keys"**

So, when the system resolves:

**/home/steen/keys**

it first reaches **n6**.

Then it reads the name stored in **n6** and continues with:

**/keys**

Finally, **/keys** leads to **n5**.

# ! Mounting Issue

Mounting is used to merge different **name spaces** transparently.  
It connects a node in the current name space to another name space.

## Terminology

### Foreign name space:

The name space we want to access.

### Mount point:

The **node in the current name space** that points to the foreign name space.

### Mounting point:

The **node in the foreign name space** where name resolution continues.

## Mounting Across a Network

To mount a remote name space, we need:

1. The access protocol. `nfs://flits.cs.vu.nl/home/steen`
2. The server name.
3. The mounting point in the foreign name space.

## Name-Space Implementation

### Basic Issue

Name resolution and name-space management should be distributed across multiple machines.  
This is done by distributing the nodes of the naming graph.

### Three Levels

#### Global level:

Contains **high-level directory nodes**.

These nodes are usually managed by different administrations. ex : .com, .edu, .tr

#### Administrative level:

Contains **mid-level directory nodes**.

They can be grouped and assigned to separate administrations.

#### Managerial level:

Contains **low-level directory nodes inside one administration**.

The main issue is mapping directory nodes to local name servers. ex : web.aydin.edu.tr

## DNS Partitioning Example

The DNS name space can be divided into three layers:

#### Global layer:

Top-level domains such as **com, edu, gov, org, net**.

#### Administrative layer:

Organizations or domains under the global layer, such as **oracle, yale, acm, vu**.

#### Managerial layer:

Lower-level nodes inside an organization, such as departments, files, or services.

# Name-Server Comparison

| Feature             | Global    | Administrative | Managerial   |
|---------------------|-----------|----------------|--------------|
| Geographical scale  | Worldwide | Organization   | Department   |
| Number of nodes     | Few       | Many           | Vast numbers |
| Responsiveness      | Seconds   | Milliseconds   | Immediate    |
| Update propagation  | Lazy      | Immediate      | Immediate    |
| Number of replicas  | Many      | None or few    | None         |
| Client-side caching | Yes       | Yes            | Sometimes    |

## Iterative Name Resolution

In **iterative name resolution**, the client contacts each name server **itself**.

- Each server resolves one part of the name.
- Then it returns the next server's address to the client.
- The client continues with the next server.

```
Client → root server
Client → nl server
Client → vu server
Client → cs server
```

## Recursive Name Resolution

In **recursive name resolution**, the client contacts **only the first name server**.

- Each server forwards the request to the next server.
- The final result returns back through the servers.
- The first server sends the result to the client.

```
Client → root server
root → nl
nl → vu
vu → cs
sonuç geri döner
```

## Caching in Recursive Name Resolution

- Servers can cache intermediate results.
- Caching reduces **repeated** lookups and improves performance.

## Scalability Issues

### Size Scalability

Servers must handle many requests per time unit.

This is a problem for high-level servers.

### Solution

Use **replication** at global and administrative levels.

Nodes can be mapped to multiple servers, and name resolution can start at the nearest server.

### Observation

Many nodes store an **address** of the entity.

But if the entity is mobile, replicated addresses may become outdated.

So, traditional name servers are not good for mobile entities.

## Scalability Issues

### Geographical Scalability

Name resolution must work across large geographical distances.

### Problem

If name servers are located **far away**, name resolution creates a **location dependency**.

This can make communication slower.

### Main Idea

- **Iterative resolution:** the client may contact many distant servers.
- **Recursive resolution:** servers forward the request between themselves.
- Long-distance communication **can increase latency**.

Name servers should be placed carefully.

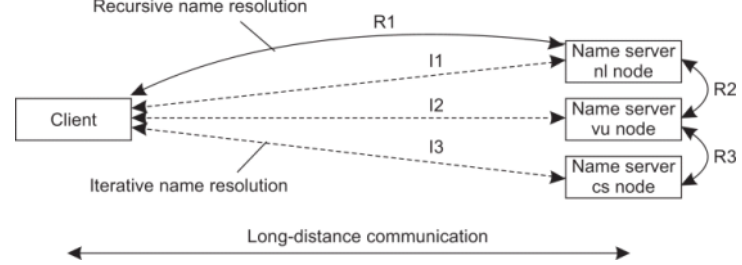
Otherwise, resolving names over large distances becomes inefficient.

# DNS

DNS is a hierarchical name space.

A **domain** is a subtree.

A **domain name** is a path name to the root of that domain.

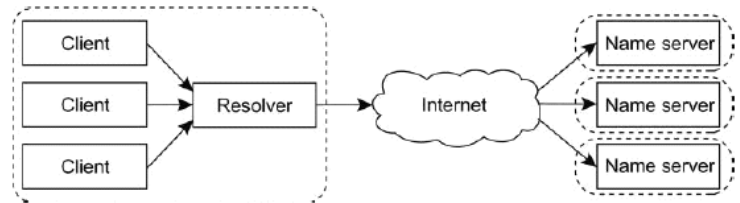


## Traditional DNS

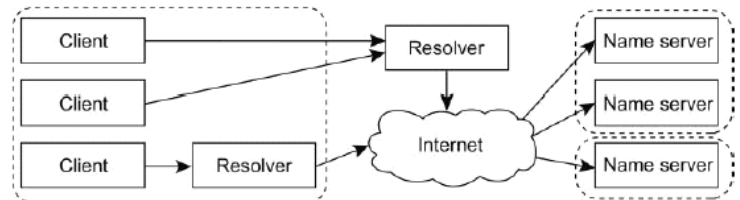
In traditional DNS, many clients use the same local **resolver**. The resolver sends DNS requests to name servers through the Internet.

## Modern DNS

In modern DNS, clients may use different resolvers. Some resolvers can be outside the local network. Improves scalability and performance.



The traditional organization of the implementation of DNS



The modern organization of DNS

## Secure DNS

### Basic Approach

In Secure DNS, resource records are grouped into a **signed set** for each zone.

Examples:

- IPv4 address records of a zone
- IPv6 address records of a zone
- Name server records of a zone

A **zone-signing key** is used to sign these record sets.

### Trusting Signatures

Zone-signing keys are also grouped and signed by another key.

This key is called the **key-signing key**.

The parent zone stores and signs the hash of this key.

So, trust can move from parent zone to child zone.

## Naming in NFS

In NFS, a server can **export** part of its file system.

Clients can **import** it by **mounting**.

Different clients can mount the same exported directory in different places, so they may have different **nonsharable name spaces**.

## Attribute-Based Naming

### Observation

Sometimes it is easier to name and find entities by their **attributes**.

This is like traditional directory services or **yellow pages**.

### Problem

Lookup can be expensive.

Because the system must compare the requested attribute values with the actual attribute values.

## Implementing Directory Services

### Scalable Searching

For scalable search, a directory service can be implemented like a **database**. It can also be combined with a structured naming system.

### LDAP (Lightweight Directory Access Protocol)

**LDAP** is used to access and search directory information.

Each directory entry has:

- **attribute-value pairs**
- a unique name



So, LDAP helps us find entities by using their attributes.

### LDAP

#### Directory Information Base (DIB):

All directory entries in an LDAP service.

#### Relative Distinguished Name (RDN):

A unique naming attribute for identify a record.

#### Directory Information Tree (DIT):

The LDAP naming tree.

Each node represents a directory entry.

### Example

An LDAP path can look like this:

**C = NL → O = VU University → OU = Computer Science → CN = Main server**

This means:

- Country is Netherlands.
- Organization is VU University.
- Unit is Computer Science.
- Common name is Main server.

So, LDAP stores entries in a tree structure and finds them by using naming attributes.

### Example

A server can be described with attributes like:

- Country: NL
- Locality: Amsterdam
- Organization: VU University
- Organizational Unit: Computer Science
- Common Name: Main server
- Mail servers, FTP server, WWW server

## Distributed Index

### Basic Idea

A distributed index maps **attribute values** to different servers.

For example, an attribute can be:

- lastName
- firstName
- price
- location

Each attribute value is handled by a specific server.

So, the client can ask the correct server for matching entities.

## Drawbacks of Distributed Index

### Problems

- A query with **k attributes** may require contacting **k servers**.
- Some attributes can return too many results.
  - Example: many people may have the last name **Smith**.
- Range queries are not easy. Example: **price = 1000–2500**

### Short Meaning

Distributed index helps attribute-based search, but complex queries can be expensive.

## Query Idea

A query can combine multiple attributes with logical operators like **AND** and **OR**.

Example:

**lastName = Smith AND firstName = Pheriby**

The client gets results from different servers and combines them.

**Named-data networking**

NDN retrieves **data by name**, not by address. The network uses the name to route the request to a place where the data is stored.